Don't Reinvent the Wheel: Efficient Instruction-Following Text Embedding based on Guided Space Transformation

Yingchaojie Feng[1]*, Yiqun Sun[2]*, Yandong Sun[2], Minfeng Zhu[3]†, Qiang Huang[4]†, Anthony K. H. Tung[2], Wei Chen[1]

[1]State Key Lab of CAD&CG, Zhejiang University
[2]School of Computing, National University of Singapore [3]Zhejiang University
[4]School of Intelligence Science and Engineering, Harbin Institute of Technology (Shenzhen)
{fycj, minfeng_zhu, chenvis}@zju.edu.cn,
{sunyq, yandong, atung}@comp.nus.edu.sg, huangqiang@hit.edu.cn

Abstract

In this work, we investigate an important task named instruction-following text embedding, which generates dynamic text embeddings that adapt to user instructions, highlighting specific attributes of text. Despite recent advancements, existing approaches suffer from significant computational overhead, as they require re-encoding the entire corpus for each new instruction. To address this challenge, we propose **GSTransform**, a novel instruction-following text embedding framework based on **G**uided **S**pace **Transform**ation. Our key observation is that instruction-relevant information is inherently encoded in generic embeddings but remains underutilized. Instead of repeatedly encoding the corpus for each instruction, GSTransform is a lightweight transformation mechanism that adapts pre-computed embeddings in real time to align with user instructions, guided by a small amount of text data with instruction-focused label annotation. We conduct extensive experiments on three instruction-awareness downstream tasks across nine real-world datasets, demonstrating that GSTransform improves instruction-following text embedding quality over state-of-the-art methods while achieving dramatic speedups of 6∼300× in real-time processing on large-scale datasets. The source code is available at https://github.com/YingchaojieFeng/GSTransform.

1 Introduction

Text embedding (Mikolov et al., 2013; Pennington et al., 2014; Zhuo et al., 2023; Li and Li, 2024) is a fundamental NLP problem, serving as the backbone of numerous applications such as clustering (Aggarwal and Zhai, 2012), Semantic Textual Similarity (STS) (Agirre et al., 2012, 2013), and information retrieval (Karpukhin et al., 2020; Thakur et al., 2021). The core objective of text embedding models is to convert textual data into fixed-length vector representations that encode semantic relationships, ensuring that similar texts remain close in the embedding space while dissimilar ones are well separated (Muennighoff et al., 2023).

Despite their widespread use, generic embedding models are inherently static and inflexible, often failing to adapt to task-specific requirements. As illustrated in Figure 1, real-world text data frequently encapsulates multiple semantic aspects (e.g., "intent" and "emotion"), yet traditional embeddings primarily focus on general semantics rather than user-specified perspectives. This limitation restricts their applicability in scenarios where users seek embeddings tailored to specific needs.

To address this challenge, recent research has introduced instruction-following text embedding, which generates embeddings in different semantic spaces conditioned on user-provided instructions (Su et al., 2023; Peng et al., 2024). These models allow users to emphasize specific aspects of text, dynamically adjusting embeddings based on task-oriented instructions. Notably, InstructOR (Su et al., 2023) concatenates instruction with text as input and fine-tunes a Transformer-based model across a diverse set of instructions to capture instruction-aware semantics. Alternatively, InBedder (Peng et al., 2024) treats instructions as questions and encodes the corresponding answers as the final embeddings, leveraging both strengths of generative models and embedding models.

Although these methods improve instruction-awareness, they suffer from significant efficiency limitations when applied to large-scale datasets. The primary issue stems from their dependence on full corpus re-encoding for each instruction. Given m instructions and n texts, both InstructOR and InBedder require $O(m \times n)$ forward passes to generate embeddings, making it computationally expensive and impractical for real-world applications

*Yingchaojie Feng and Yiqun Sun contributed equally to this work.
†Minfeng Zhu and Qiang Huang are corresponding authors.

Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 24511–24525
July 27 - August 1, 2025 ©2025 Association for Computational Linguistics

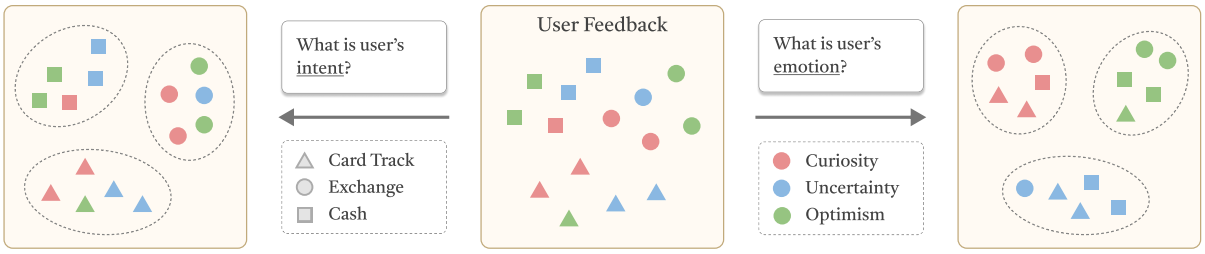


Figure 1: An illustration of instruction-following text embeddings. Depending on different instructions, the embeddings reorganize their similarity relationships, forming distinct clusters that emphasize different aspects.

where pre-encoded embeddings are stored in vector databases. This inefficiency hinders experimentation with different instructions, as even minor changes require reprocessing the entire dataset.

On the other hand, we observe that generic text embeddings already contain latent instruction-relevant information but do not prioritize it explicitly. Instead of re-encoding the full corpus for every new instruction, we propose a transformation-based approach that dynamically adjusts existing embeddings to align with user-specified instructions. As illustrated in Figure 1, different users may prioritize distinct semantic attributes. For instance, when the focus is on the "intent" aspect, the transformation highlights the shape of embeddings to cluster texts based on their underlying purpose. Conversely, when the emphasis shifts to the "emotion" aspect, the transformation accentuates color, grouping texts by their sentiment.

To this end, we introduce **GSTransform**, a novel instruction-following text embedding framework based on **G**uided **S**pace **Transform**ation, eliminating the need for exhaustive re-encoding. GSTransform consists of two components:

- **Instruction-based Label Construction:** We build a label taxonomy from user instructions to categorize text representations according to instruction-specific semantics.
- **Label-guided Embedding Transformation:** We adapt the original embedding space using these instruction-driven labels to align with user-specified information.

Rather than regenerating embeddings from scratch, GSTransform treats instructions as transformation operators that strategically reorient attention within the existing semantic space, ensuring adaptability without excessive computation. The instruction-driven label data guides the training of transformation models, providing a clearer optimization objective than direct training on instructions.

Contributions. This work makes the following essential contributions:

(1) **Efficient Instruction-Following Embedding:** We propose GSTransform, a novel framework that enables instruction-following text embeddings via guided space transformation, eliminating the need for re-encoding the entire corpus for each instruction.

(2) **Cost-Effective Instruction Adaptation:** GSTransform introduces a Label-guided Embedding Transformation mechanism that adapts pre-computed embeddings to user instructions using a lightweight model trained on a small, annotated subset, requiring only a fixed number of LLM calls, making it highly cost-effective for large-scale applications.

(3) **Comprehensive Empirical Validation:** We conduct extensive experiments on three instruction-awareness tasks across nine real-world datasets, showing that GSTransform improves embedding quality while significantly reducing computational overhead and latency on large-scale datasets compared to state-of-the-art baselines.

2 Related Work

2.1 Generic Text Embedding

Text embedding has been a long-studied problem. Since word embeddings, people adopt self-supervised training in generating word embeddings, and pool the word embeddings to form text embeddings (Mikolov et al., 2013; Pennington et al., 2014). Recent advancements in context-aware semantic text embedding models leverage Transformer-based architectures (Vaswani et al., 2017; Devlin et al., 2019) as their backbone, often employing customized objectives like contrastive loss to train the models (Cer et al., 2018; Reimers and Gurevych, 2019; Gao et al., 2021; Zhuo et al., 2023). Moreover, state-of-the-art (SOTA) text embedding models have been further enhanced with techniques such as using large language models (LLMs) (Wang et al., 2023; Muennighoff et al.,

2025; Lei et al., 2024) and more sophisticated loss functions designed to address issues like cosine saturation (Li and Li, 2024).

Despite their effectiveness, these methods lack generalizability and fail to meet diverse user needs when downstream tasks require focusing on specific aspects beyond general semantics.

2.2 Instruction-Following Text Embedding

Instruction-following text embedding (Su et al., 2023; Peng et al., 2024) allows users to guide embedding generation through customized instructions. The model produces embeddings that align with users' specific interests by considering both the input text and instructions.

InstructOR (Su et al., 2023) pioneered instruction-based embeddings by concatenating instructions with input texts and training the model using contrastive objectives across a diverse set of instructions. It adapts embeddings for varied semantic interpretations but does not explicitly model instruction-specific semantic aspects. InBedder (Peng et al., 2024) extends this idea by treating instructions as questions and generating intermediate answers to produce more fine-grained, instruction-aware embeddings. They also propose Instruction Awareness Tests, which we adopt to evaluate Triplet Alignment, STS, and Clustering tasks. Yet, both methods require re-encoding the entire corpus for each new instruction, resulting in notable computational overhead and latency, especially for large-scale datasets.

Beyond text embeddings, related efforts have explored instruction-aware and prompt-based information retrieval (Weller et al., 2025b; Min et al., 2025; Oh et al., 2024; Sun et al., 2024; Weller et al., 2025a), offering alternative formulations that leverage user intent to enhance retrieval quality.

3 The GSTransform Framework

We introduce GSTransform, a novel framework designed for efficient instruction-following embedding generation, enabling dynamic adaptation of text embeddings to user-specified instructions. In contrast to prior methods that require re-encoding the entire corpus for every instruction, GSTransform operates directly on pre-computed embeddings, allowing real-time transformation with minimal computational overhead.

As illustrated in Figure 2, GSTransform comprises two core components:

(1) **Instruction-based Label Construction:** This component constructs an instruction-aware label taxonomy to guide downstream transformation. A small corpus of texts is randomly sampled, and instruction-guided summaries are generated using an LLM. These summaries help to capture the key semantic aspects emphasized by the instruction, enabling the taxonomy to reflect fine-grained, instruction-relevant distinctions.

(2) **Label-guided Embedding Transformation:** Once the label taxonomy is constructed, we use an LLM to annotate the sampled texts according to the taxonomy. The resulting labeled data is then used to train a lightweight Embedding Transformation Model, which maps generic embeddings to an instruction-aligned semantic space. The model jointly optimizes a contrastive loss, which is used to highlight instruction-specific similarities, and a reconstruction loss, which aims to preserve general semantic structure.

In all LLM-involved stages (summarization, labeling, classification), we use GPT-4o-mini for its balance of performance and efficiency, and the prompt templates are provided in Appendix A. By decoupling instruction adaptation from embedding generation, GSTransform supports scalable, real-time transformations without the need for repeated re-encoding. We now begin by detailing the first component: Instruction-based Label Construction.

3.1 Instruction-based Label Construction

To ensure that the label taxonomy aligns with user-specified instructions while remaining sensitive to dataset-specific nuances, we adopt a bottom-up construction strategy. Rather than processing the entire corpus, which is often computationally expensive for large corpora, we randomly sample a representative subset (e.g., 3,000 texts in our experiments) for label construction. While instruction-aware labeled data can be pre-defined, this component addresses the common scenario where such labeled data is unavailable or costly to obtain.

Instruction-Following Clustering. Traditional label construction methods, such as BERTopic (Grootendorst, 2022), typically involve clustering embeddings obtained from generic models, followed by heuristic label generation. However, these methods often fail to capture instruction-specific distinctions, as generic embeddings do not prioritize

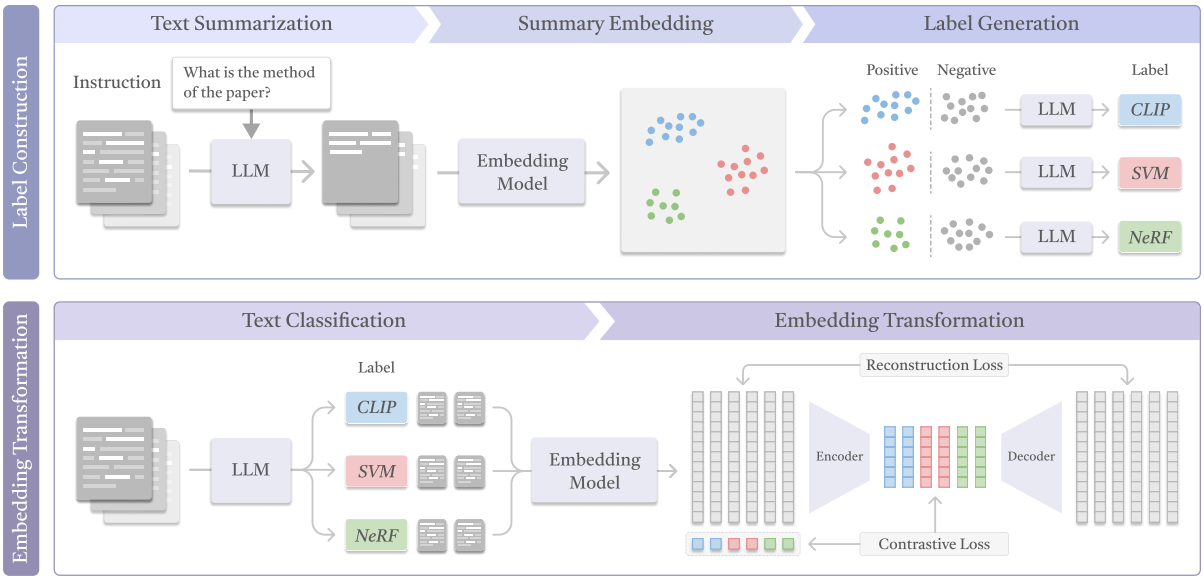


Figure 2: GSTransform consists of two core components: (1) Instruction-based Label Construction, which summarizes the texts to extract instruction-relevant information, performs embedding clustering on summary text, and generates labels to represent their characteristics; (2) Label-Guided Embedding Transformation, which classifies the text based on the constructed labels and transforms the generic embedding space guided by the labeled data.

user-specified attributes.

To overcome this limitation, we introduce Instruction-following Clustering, a method designed to extract and amplify instruction-relevant semantics from the input text. The process comprises three key steps:

- **Step 1: Instruction-Guided Summarization.** We first use an LLM to generate concise summaries of each sampled text, tailored to the user's instructions.
- **Step 2: Embedding with Generic Models.** The instruction-guided summaries are then encoded using a generic embedding model, such as UAE (Universal AnglE Embedding) (Li and Li, 2024).
- **Step 3: Clustering.** The resulting summary embeddings are grouped using the k-means++ algorithm (Arthur and Vassilvitskii, 2007), producing clusters that align with instruction-specified aspects.

The number of clusters (k) determines the granularity of the label taxonomy. We set $k = 50$ as a default, which we found to offer a good balance between semantic specificity and generalizability without extensive hyperparameter tuning. The effect of k is studied in Section 4.6.

Label Generation. To semantically characterize each cluster, we generate representative labels using LLMs. However, due to the non-deterministic nature of LLM outputs, the generated labels may vary in quality, ranging from *overly generic* (failing to distinguish between clusters) to *overly specific* (lacking generalizability beyond the sampled texts).

To address this, we adopt a contrastive prompting strategy inspired by principles in contrastive learning. Specifically, each LLM prompt is composed of both *positive examples* (texts sampled from within the target cluster) and *negative examples* (texts drawn from other clusters), encouraging the LLM to focus on the discriminative features that define the cluster while being aware of what sets it apart from others.

This strategy ensures that the generated labels are: semantically aligned with the instruction-relevant focus, mutually exclusive across clusters, and generalizable to unseen examples beyond the sampled subset. As a result, the generated labels serve as reliable supervision signals for training the embedding transformation model in the next stage.

3.2 Label-Guided Embedding Transformation

This component transforms the original embedding space into a new space that focuses on instruction-specific information. We first annotate the sample text using the generated label taxonomy and then train a transformation model to project generic embeddings into an instruction-adapted space.

Text Classification. In this step, we use the generated label taxonomy to annotate sample data for training the transformation model. Given the pow-

erful capability of LLMs in semantic understanding and instruction following, we employ LLMs to classify the sampled text based on user instructions and the generated label taxonomy. We classify the original text instead of the summary to leverage the rich context of the original text.

Embedding Transformation. The transformation model aims to restructure the embedding space to emphasize instruction-relevant features while preserving general semantic integrity. Once trained, it can dynamically transform any embedding from the generic space into an instruction-specific space, enabling efficient large-scale processing without requiring access to the original texts.

Specifically, the transformation model adopts a lightweight encoder-decoder architecture, where both the encoder and decoder are implemented as *single-layer linear models*. This simple yet effective design avoids overfitting and ensures that the transformation preserves the overall structure of the original embedding space.

The encoder transforms the each input embedding vector ($\boldsymbol{x}_i$) into an instruction-aware representation ($\boldsymbol{e}_i$), while the decoder reconstructs the original embedding ($\hat{\boldsymbol{x}}_i$) from the transformed vector ($\boldsymbol{e}_i$). The model is trained with a dual-objective loss function that balances semantic adaptation and information preservation:

$$\mathcal{L} = \beta_1 \cdot \mathcal{L}_{contr} + \beta_2 \cdot \mathcal{L}_{recon},$$

where β_1 and β_2 are weight coefficients (both set to 1.0 by default).

The **contrastive loss** $\mathcal{L}_{contr}$ encourages embeddings of texts sharing the same label (as defined by the instruction-based taxonomy) to be closer, while separating those with different labels:

$$\mathcal{L}_{contr} = \frac{1}{|N|^2} \sum_{i,j \in N} \Big(\mathbf{1}_{y_i = y_j} \cdot D(\boldsymbol{e}_i, \boldsymbol{e}_j)^2 + \mathbf{1}_{y_i \neq y_j} \cdot D'(\boldsymbol{e}_i, \boldsymbol{e}_j)^2 \Big),$$

where D represents the Euclidean distance, and $D'(\boldsymbol{e}_i, \boldsymbol{e}_j) = \max(0, m - D(\boldsymbol{e}_i, \boldsymbol{e}_j))$ incorporates a margin m to enforce separation between classes. $\mathbf{1}_{y_i = y_j}$ is an indicator that equals 1 if $\boldsymbol{e}_i$ and $\boldsymbol{e}_j$ belong to the same class and 0 otherwise.

The **reconstruction loss** $\mathcal{L}_{recon}$ minimizes information loss by penalizing the differences between the reconstructed $\hat{\boldsymbol{x}}_i$ and original embeddings $\boldsymbol{x}_i$:

$$\mathcal{L}_{recon} = \frac{1}{N} \sum_{i=1}^{N} \|\hat{\boldsymbol{x}}_i - \boldsymbol{x}_i\|^2.$$

To train the model, we use the instruction-annotated subset (e.g., 3,000 samples), splitting it into 80% training and 20% validation. We apply early stopping based on validation loss to ensure efficient convergence and avoid overfitting. Once trained, the model can be applied to transform any generic embedding into an instruction-aligned representation, enabling scalable and real-time adaptation without requiring access to the original text.

4 Experiments

In this section, we conduct a comprehensive evaluation of GSTransform to assess its effectiveness, efficiency, and robustness across a range of instruction-aware embedding tasks. In particular, our experiments are designed to answer the following research questions (RQs):

- **RQ1 (Effectiveness):** Can GSTransform improve the instruction-following capability of generic embedding models across diverse tasks and datasets? (Section 4.3)
- **RQ2 (Efficiency):** How does GSTransform compare to state-of-the-art instruction-aware embedding baselines in terms of runtime and cost? (Section 4.4)
- **RQ3 (Component Contribution):** What is the contribution of each individual component in the GSTransform framework? (Section 4.5)
- **RQ4 (Robustness):** How robust is GSTransform to variations in hyperparameters, such as sample size and clustering granularity? (Section 4.6)

To complement the quantitative results, we also include a Case Study (Section 4.7) to qualitatively illustrate how GSTransform adapts embedding structures to different instruction foci.

4.1 Baseline Models

In the experiments, we evaluate GSTransform on three prevalent embedding models: **UAE** (Universal AnglE Embedding) (Li and Li, 2024), **Mxbai** (Lee et al., 2024), and **BGE** (BAAI General Embedding) (Xiao et al., 2023). These models, designed for generic embeddings, inherently lack instruction-following capabilities.

To assess the impact of GSTransform, we examine how well it enhances their ability to follow user instructions. For comparison, we benchmark GSTransform against three state-of-the-art instruction-aware baselines: **InstructOR** (`instructor-large`) (Su et al., 2023),

InBedder-Roberta (`roberta-large-InBedder`), and **InBedder-Llama2** (`llama-2-7b-InBedder`) (Peng et al., 2024). For a fair evaluation, we use default open-source configurations for all models.

4.2 Downstream Tasks and Datasets

To evaluate embedding performance in an instruction-aware setting, we follow the protocol established in InBedder (Peng et al., 2024) and assess GSTransform on three representative downstream tasks: Clustering, Semantic Textual Similarity (STS), and Triplet Alignment. For each task, we employ three diverse datasets, each capturing either single or multi-dimensional semantic attributes. For instance, the Papers with Codes (PaperCode) dataset (Ostendorff et al., 2022) contains both a *task* and a *method* dimension, enabling evaluation from multiple perspectives.

We craft user instructions to explicitly highlight the target dimensions. The instructions for baselines follow their default stylistic conventions: command-style instructions for InstructOR and question-style for InBedder. Both styles preserve the same semantic intent. GSTransform also uses question-style instructions. Full task and dataset details are summarized in Appendix B, and the corresponding instructions are detailed in Appendix C. Below, we briefly describe the three tasks.

Clustering. This task evaluates how well the structure of the embedding space aligns with the semantic focus of the instruction. We assess clustering performance on three datasets: NYT-Clustering (**NYTClust**) (Peng et al., 2024), Amazon CounterFactual (**AmzCF**) (O'Neill et al., 2021), and MasakhaNews (**MNews**) (Adelani et al., 2023). Clustering quality is measured using the **V-measure** (Rosenberg and Hirschberg, 2007), which quantifies the consistency between predicted clusters and ground-truth labels. For NYTClust, which includes multiple aspects, we report the harmonic mean across aspect-specific results.

Semantic Text Similarity (STS). This task examines whether embeddings of sentence pairs correctly reflect their semantic similarity under the instruction-defined perspective. We use three datasets for this task: **PaperCode** (Ostendorff et al., 2022), Multi-HateCheck (**MultiHate**) (Röttger et al., 2022), and **Big Patent** (Sharma et al., 2019). Following (Peng et al., 2024), we sample 50,000 sentence pairs per instruction from annotated datasets. Each pair is then assigned a binary label (1 if they share the same class, 0 otherwise) (Peng et al., 2024). Finally, we compute **Spearman correlation** between the binary labels and the cosine similarities of the embedding pairs. Since the Spearman correlation can be negative, we report the arithmetic mean for datasets with multiple aspects.

Triplet Alignment. This task evaluates whether the embeddings capture relative similarity across three texts: an anchor, a positive, and a negative. A correct alignment requires that the distance between the anchor and positive be smaller than that between the anchor and negative. We use three datasets for this task: IntentEmotion (**IntEmo**) (Peng et al., 2024), Toxic_conversations_50k (**Toxic**) (Do, 2019), and **AG-News** (Zhang et al., 2015). For each dataset, we randomly sample 50,000 triplets, where the anchor and positive share the same label, and the negative has a different label. We report **Triplet Alignment Accuracy** as the proportion of triplets where this relative ordering is correctly captured. For multi-aspect datasets like IntEmo, we report the harmonic mean across aspects.

4.3 Main Results

Table 1 presents the performance comparison results of GSTransform against baseline models. The strongest baseline, InBedder-Llama2, achieves an average score of 55.31 across the datasets. In contrast, GSTransform significantly outperforms all baselines, achieving an average score of 66.01, demonstrating its effectiveness in improving instruction-following embeddings. A particularly notable gain is observed on the AmzCF dataset, where GSTransform improves performance from 1.49 to 34.68, highlighting its ability to extract instruction-relevant information from generic embeddings. These results suggest that our approach effectively enhances the instruction-aware clustering quality by aligning the embedding space with user-specific instructions.

Moreover, GSTransform improves the instruction following capability of generic embedding models such as UAE, Mxbai, and BGE. For instance, applying GSTransform to BGE enhances performance on NYTClust from 55.90 to 75.60. The consistent gains across different backbone models demonstrate the robustness of our approach in enhancing instruction-aware embeddings.

We also notice that the performance gains of GSTransform vary across tasks. We believe this can be partially attributed to the degree of alignment

Model	Clustering (V-measure ↑)			STS (Spearman Corr. ↑)			Triplet Alignment (Accu. ↑)			Mean ↑
	NYTClust	AmzCF	MNews	PaperCode	MultiHate	Big Patent	IntEmo	Toxic	AG-News	
InstructOR	53.88	1.49	60.33	55.53	36.46	21.99	51.96	53.75	73.41	45.42
InBedder-Roberta	58.47	0.40	45.17	43.82	33.27	24.44	92.64	51.49	80.99	47.85
InBedder-Llama2	72.70	1.35	61.14	46.65	45.67	37.81	90.18	55.41	86.93	55.31
UAE	52.72	1.63	62.24	64.53	37.73	21.64	36.25	55.20	72.59	44.95
GSTransform (UAE)	73.66	**34.68**	63.47	83.14	53.17	38.05	96.72	63.01	87.13	65.89
Relative Gain	+20.94	+33.05	+1.23	+18.61	+15.44	+16.41	+60.46	+7.81	+14.54	+20.94
Mxbai	54.95	0.13	63.09	64.85	38.10	22.84	35.37	55.67	75.41	45.60
GSTransform (Mxbai)	73.92	34.14	**64.71**	**83.43**	52.45	**38.34**	96.30	**63.53**	**87.27**	**66.01**
Relative Gain	+18.98	+34.01	+1.62	+18.58	+14.35	+15.50	+60.94	+7.86	+11.86	+20.41
BGE	55.90	0.13	47.09	65.04	37.84	16.16	42.83	54.88	71.46	43.48
GSTransform (BGE)	**75.60**	33.04	63.35	83.13	**53.24**	34.56	**96.75**	61.20	87.01	65.32
Relative Gain	+19.69	+32.91	+16.26	+18.09	+15.40	+18.40	+53.93	+6.32	+15.55	+21.84

Table 1: Experimental results on three downstream tasks with their nine associated datasets.

between the datasets and pretraining data of (large) language models. For widely used datasets like AG-News and NYTClust, baseline models have likely been pre-trained on similar or overlapping content, leaving limited headroom for further improvements. In contrast, datasets such as Big Patent involve domain-specific terminology and complex patent structures that general-purpose LLMs may not represent well. This semantic mismatch can reduce the effectiveness of instruction-based adaptation, resulting in smaller relative gains.

4.4 Computational Efficiency

To assess computational efficiency, we compare GSTransform with the three baselines on two large-scale datasets (AG-News and Big Patent) in terms of time and cost. AG-News contains 127.6K texts with character lengths ranging from 100 to 1010, while Big Patent consists of 67.1K texts with lengths varying between 2.76K to 3.11M. Big Patent comprises 67.1K text with character lengths ranging from 2.76K to 3.11M. All models are evaluated on a server equipped with an NVIDIA A100 GPU (80GB memory) to ensure fairness.

Time and Cost Efficiency. For time evaluation, we measure the *embedding generation time* for three different instructions. Unlike baselines that require re-encoding the entire dataset per instruction, GSTransform supports one-time pre-computing generic embeddings and applying real-time transformations, significantly reducing latency. Thus, we separately calculate the time for pre-computing and real-time transformation. For cost evaluation, we calculate the server rental cost (1 dollar per hour) and OpenAI API call costs.

Result Analysis. Table 2 presents the results: Baseline models exhibit scalability issues, as their real-time latency is directly proportional to dataset size. Notably, InBedder-Llama2 suffers from severe inefficiencies, requiring 29,756 seconds due to its question-answering generation pipeline. In contrast, GSTransform processes only a small annotated subset (3,000 texts), reducing real-time latency to just 87 seconds, making it 6$\sim$300$\times$ faster than existing methods. We also compare the real-time latency of these models at different scales. The results (see Appendix E) show baseline models exhibit latency that increases proportionally with data size, making them more suitable for small-scale datasets. In contrast, GSTransform maintains near-constant latency, since its transformation is learned from a fixed sample size. This results in clear efficiency gains on large-scale datasets.

In terms of cost, GSTransform achieves state-of-the-art performance at a cost comparable to smaller models, making it a scalable and economical solution for real-world applications. Additionally, its flexibility enables seamless integration with future advanced and cost-efficient LLMs.

4.5 Ablation Studies

We conduct ablation studies to assess the contributions of key components in GSTransform. For Instruction-based Label Construction, we design two alternative solutions: (1) Removing the instruction-based text summarization component and (2) Replacing the entire process with directed LLM-based label generation. For Label-guided Embedding Transformation, we replace the transformation model with Fisher Discriminant Analysis (FDA) (Mika et al., 1999). We also assess different embedding backbones to test the generalizability of our method. The results for three datasets are shown in Table 3, while full results for all datasets

Model	AG-News			Big Patent		
	Pre. Time (s) ↓	RT-Latency (s) ↓	Est. Cost ($) ↓	Pre. Time (s) ↓	RT-Latency (s) ↓	Est. Cost ($) ↓
InstructOR	-	511	0.43	-	1,679	1.40
InBedder-Roberta	-	749	0.62	-	1,781	1.48
InBedder-Llama2	-	16,206	13.51	-	29,756	24.80
GSTransform	626	77	0.96	1,689	87	2.37

Table 2: Computational efficiency results on two large-scale datasets. We report the Pre-computing Time (Pre. Time), Real-Time Latency (RT-Latency, per instruction), and Estimated Cost (for 3 instructions) for each model.

Solutions	Dataset			Mean ↑
	NYTClust	MultiHate	IntEmo	
GSTransform (Mxbai)	73.92	52.45	96.30	74.22
Remove Summ.	68.32	16.21	85.14	56.56
Directed Label Gen.	42.50	20.55	86.76	49.94
FDA-based Transf.	67.53	48.49	89.19	68.40

Table 3: The performance comparison of different solutions in ablation studies.

Dataset	# Samples				
	1,000	2,000	3,000	4,000	5,000
AmzCF (Clustering)	28.14	32.25	34.14	34.61	35.72
MultiHate (STS)	49.04	50.65	52.45	54.13	54.64
Toxic (Trip. Align.)	61.80	63.09	63.53	63.88	64.20
Mean	46.33	48.66	50.04	50.87	51.52

Table 4: The impact of the number of samples.

Dataset	# Clusters (k)				
	10	30	50	70	90
AmzCF (Clustering)	35.37	33.95	34.14	34.87	33.46
MultiHate (STS)	52.12	52.69	52.45	52.51	52.46
Toxic (Trip. Align.)	63.59	64.17	63.53	64.05	63.31
Mean	50.36	50.27	50.04	50.48	49.74

Table 5: The impact of the number of clusters (k).

are provided in Appendix D.

Remove Summarization. Removing instruction-based text summarization leads to a significant performance drop (74.22 → 56.56), particularly affecting datasets like MultiHate. Further analysis reveals that generic embeddings *fail* to distinguish between hateful and non-hateful texts, causing the non-hateful category to be overwhelmed in clustering, reinforcing the importance of instruction-guided summarization.

Directed Label Generation. Eliminating the text summarization, summary clustering, and label generation steps and directly relying on LLMs for label generation results in an even steeper performance decline (74.22 → 49.94). We observe that LLM-generated labels often fail to align with dataset characteristics, leading to suboptimal text classification and embedding transformations. This suggests that context-aware taxonomy construction is crucial for effective instruction adaptation.

FDA-based Transformation. Replacing the encoder-decoder architecture with FDA projection decreases performance (74.22 → 68.40), confirming that our model better preserves instruction-relevant semantic structure compared to traditional dimensionality reduction techniques.

Generalization Across Embedding Models. We test GSTransform on different generic embedding models (UAE, Mxbai, and BGE), and the results in Table 1 show consistent improvements (44.95 → 65.89, 45.60 → 66.01, and 43.48 → 65.32), verifying the method's robustness and model-agnostic adaptability.

4.6 Parameter Studies

To assess the robustness of GSTransform to key hyperparameters, we conduct a sensitivity analysis on three representative datasets–Toxic, AmzCF, and MultiHate. Specifically, we vary: (1) the number of samples for transformation model training, and (2) the number of clusters (k) for label taxonomy.

Number of Samples. We vary the number of training samples from 1,000 to 5,000 while keeping all other settings fixed. Using Mxbai as the embedding backbone, we report the results in Table 4. The results show that while increasing the sample size generally leads to improved performance, the gains plateau around 3,000 samples, indicating diminishing returns beyond this point. This finding highlights the efficiency of our method: it achieves strong performance with only a moderate amount of LLM-annotated data.

Number of Clusters. We also evaluate the impact of varying the number of clusters k in the k-means++ algorithm used for label construction. As depicted in Table 5, performance remains largely stable across a wide range of k values, suggesting that GSTransform is *robust* to variations in label granularity. This insensitivity to hyperparam-

eter tuning enhances its practicality in real-world settings, where optimal values of k may be difficult to determine a priori.

4.7 Case Study

To showcase the effectiveness of GSTransform, we conduct a case study using the UAE embedding model on the NYTClust dataset, where users aim to cluster news articles by country.

Consider a scenario where the user is interested in the location mentioned in the news. The user instructions can be "What country does the news mention?" Since the UAE model inherently does not support user instructions, the embedding results fail to clearly distinguish the news from different countries. We visualized the UAE embeddings using t-SNE projection in Figure 3A. Colors encode different ground-truth country labels. The projection shows significant overlap between countries, with some news from the same country distributed in different areas. Figure 3B visualizes the projection of the transformed embeddings based on user instruction. The clusters are separated from each other more clearly and are more aggregated inside. It is easy to identify the number of clusters and the texts in each cluster, offering a clear overview of the countries in the news dataset.

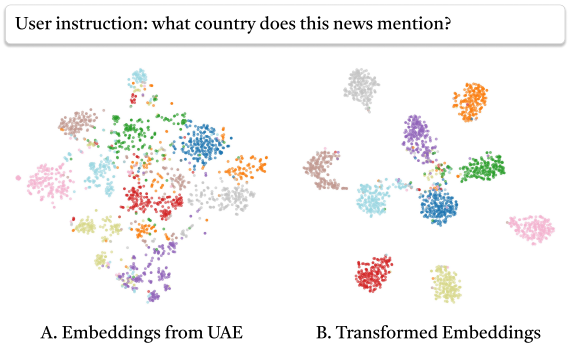


Figure 3: The embedding visualization of UAE and GSTransform (UAE). We use t-SNE for projection and encode the country labels in different colors.

We also examine the cosine distance relation for a triplet text under different instructions in Figure 4. In the triplet, the first text reports the same country (i.e., Britain) as the second text and has the same topic (i.e., education) as the third one. Our method transforms the embeddings from the UAE model (Figure 4A) based on different user instructions. When the user focuses on the countries of the news, the first and second texts are much closer (cosine distance = 0.05) as they share the same country

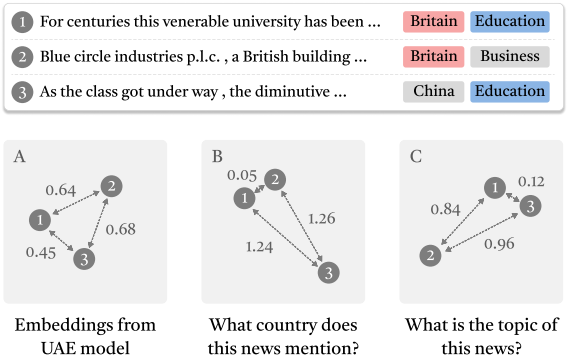


Figure 4: The cosine distance relations between a triplet text based on different user instructions.

(Figure 4B). When the instruction focus shifts to the topic of the news, the first and third texts are closer (cosine distance = 0.12) due to their common topic of education (Figure 4C). These results show that our method effectively adapts embeddings to reflect the user's focus, providing suitable distance relations that align with the instruction focus.

5 Conclusions

In this paper, we introduce GSTransform, a novel and efficient framework for generating instruction-following text embeddings via guided space transformation. Unlike prior methods that require full corpus re-encoding for every new instruction, GSTransform enables lightweight, real-time adaptation of pre-computed embeddings, significantly reducing computational overhead. The framework consists of two key components: (1) Instruction-based Label Construction, which summarizes a sampled corpus based on user instructions and clusters the resulting embeddings to capture instruction-relevant semantics; (2) Label-guided Embedding Transformation, which trains a simple encoder-decoder model to project generic embeddings into an instruction-aware semantic space using the constructed labels. Extensive experiments across nine real-world datasets and three tasks demonstrate that GSTransform consistently improves embedding quality and outperforms state-of-the-art instruction-following baselines. Notably, it achieves 6∼300× speedups in real-time processing on large datasets, underscoring its scalability and efficiency. Ablation and parameter studies further validate the effectiveness and robustness of each component. GSTransform offers a promising path toward flexible, efficient, and instruction-adaptable embeddings, unlocking new opportunities for user-customized semantic representations in real-world applications.

Limitations

Robustness of Transformation Quality. The robustness of transformation quality is influenced by several factors. First, while random text sampling for label construction is effective, it can be sensitive to data imbalance, underscoring the potential of advanced methods like coreset selection to enhance data representativeness (Chai et al., 2023). Second, the capability of generic embeddings to capture instruction-specific nuances is critical, highlighting the need for high-quality generic embeddings as a reliable foundation for transformation.

Encoder-Decoder Model Design. The current encoder-decoder architecture utilizes simple linear layers, striking a balance between performance and computational efficiency. Although this design choice is sufficient for our current objectives, future research could investigate more sophisticated architectural variants, such as incorporating non-linear transformations or attention mechanisms. These extensions should carefully weigh potential performance gains against increased computational complexity.

Acknowledgments

This research is supported by the National Natural Science Foundation of China under grant Nos. 62132017, 62302435, the "Pioneer" and "Leading Goose" R&D Program of Zhejiang (2024C01167), Zhejiang Provincial Natural Science Foundation of China under Grant No. LD24F020011, the Ministry of Education, Singapore, under its MOE AcRF TIER 3 Grant (MOE-MOET32022-0001), and the National Research Foundation, Singapore, under its AI Singapore Programme (AISG Award No. AISG3-RP-2022-029). Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the Ministry of Education, Singapore, or the National Research Foundation, Singapore.

References

David Ifeoluwa Adelani, Marek Masiak, Israel Abebe Azime, Jesujoba Alabi, Atnafu Lambebo Tonja, Christine Mwase, Odunayo Ogundepo, Bonaventure FP Dossou, Akintunde Oladipo, Doreen Nixdorf, et al. 2023. Masakhanews: News topic classification for african languages. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (AACL-IJCNLP)*, pages 144–159.

Charu C. Aggarwal and ChengXiang Zhai. 2012. A survey of text clustering algorithms. *Mining Text Data*, pages 77–128.

Eneko Agirre, Daniel M. Cer, Mona T. Diab, and Aitor Gonzalez-Agirre. 2012. Semeval-2012 task 6: A pilot on semantic textual similarity. In *Proceedings of the 6th International Workshop on Semantic Evaluation, SemEval@NAACL-HLT 2012, Montréal, Canada, June 7-8, 2012*, pages 385–393.

Eneko Agirre, Daniel M. Cer, Mona T. Diab, Aitor Gonzalez-Agirre, and Weiwei Guo. 2013. *sem 2013 shared task: Semantic textual similarity. In *Proceedings of the Second Joint Conference on Lexical and Computational Semantics, *SEM 2013, June 13-14, 2013, Atlanta, Georgia, USA*, pages 32–43.

David Arthur and Sergei Vassilvitskii. 2007. k-means++: the advantages of careful seeding. In *Proceedings of the eighteenth annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1027–1035.

Daniel Cer, Yinfei Yang, Sheng-yi Kong, Nan Hua, Nicole Limtiaco, Rhomni St. John, Noah Constant, Mario Guajardo-Cespedes, Steve Yuan, Chris Tar, Brian Strope, and Ray Kurzweil. 2018. Universal sentence encoder for english. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, pages 169–174.

Chengliang Chai, Jiayi Wang, Nan Tang, Ye Yuan, Jiabin Liu, Yuhao Deng, and Guoren Wang. 2023. Efficient coreset selection with cluster-based methods. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pages 167–178.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4171–4186.

Quan Do. 2019. Jigsaw unintended bias in toxicity classification.

Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. Simcse: Simple contrastive learning of sentence embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6894–6910.

Maarten Grootendorst. 2022. Bertopic: Neural topic modeling with a class-based TF-IDF procedure. *arXiv*.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781.

Sean Lee, Aamir Shakir, Darius Koenig, and Julius Lipp. 2024. Open source strikes bread - new fluffy embeddings model.

Yibin Lei, Di Wu, Tianyi Zhou, Tao Shen, Yu Cao, Chongyang Tao, and Andrew Yates. 2024. Meta-task prompting elicits embeddings from large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 10141–10157.

Xianming Li and Jing Li. 2024. Aoe: Angle-optimized embeddings for semantic textual similarity. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1825–1839.

Sebastian Mika, Gunnar Ratsch, Jason Weston, Bernhard Scholkopf, and Klaus-Robert Mullers. 1999. Fisher discriminant analysis with kernels. In *Neural networks for signal processing IX: Proceedings of the 1999 IEEE signal processing society workshop (cat. no. 98th8468)*, pages 41–48.

Tomás Mikolov, Ilya Sutskever, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Proceedings of the 26th International Conference on Neural Information Processing Systems (NIPS)*, pages 3111–3119.

Dehai Min, Zhiyang Xu, Guilin Qi, Lifu Huang, and Chenyu You. 2025. Unihgkr: Unified instruction-aware heterogeneous knowledge retrievers. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4577–4594.

Niklas Muennighoff, Hongjin Su, Liang Wang, Nan Yang, Furu Wei, Tao Yu, Amanpreet Singh, and Douwe Kiela. 2025. Generative representational instruction tuning. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

Niklas Muennighoff, Nouamane Tazi, Loic Magne, and Nils Reimers. 2023. MTEB: Massive Text Embedding Benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 2014–2037.

Hanseok Oh, Hyunji Lee, Seonghyeon Ye, Haebin Shin, Hansol Jang, Changwook Jun, and Minjoon Seo. 2024. INSTRUCTIR: A benchmark for instruction following of information retrieval models. *arXiv*.

James O'Neill, Polina Rozenshtein, Ryuichi Kiryo, Motoko Kubota, and Danushka Bollegala. 2021. I wish I would have loved this one, but I didn't - A multilingual dataset for counterfactual detection in product review. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7092–7108.

Malte Ostendorff, Till Blume, Terry Ruas, Bela Gipp, and Georg Rehm. 2022. Specialized document embeddings for aspect-based similarity of research papers. In *Proceedings of the 22nd ACM/IEEE Joint Conference on Digital Libraries (JCDL)*, page 7.

Letian Peng, Yuwei Zhang, Zilong Wang, Jayanth Srinivasa, Gaowen Liu, Zihan Wang, and Jingbo Shang. 2024. Answer is all you need: Instruction-following text embedding via answering the question. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 459–477.

Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543.

Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3980–3990.

Andrew Rosenberg and Julia Hirschberg. 2007. V-measure: A conditional entropy-based external cluster evaluation measure. In *Proceedings of the 2007 joint conference on empirical methods in natural language processing and computational natural language learning (EMNLP-CoNLL)*, pages 410–420.

Paul Röttger, Haitham Seelawi, Debora Nozza, Zeerak Talat, and Bertie Vidgen. 2022. Multilingual HateCheck: Functional tests for multilingual hate speech detection models. In *Proceedings of the Sixth Workshop on Online Abuse and Harms (WOAH)*.

Eva Sharma, Chen Li, and Lu Wang. 2019. BIGPATENT: A large-scale dataset for abstractive and coherent summarization. In *Proceedings of the 57th Conference of the Association for Computational Linguistics (ACL)*, pages 2204–2213.

Hongjin Su, Weijia Shi, Jungo Kasai, Yizhong Wang, Yushi Hu, Mari Ostendorf, Wen-tau Yih, Noah A. Smith, Luke Zettlemoyer, and Tao Yu. 2023. One embedder, any task: Instruction-finetuned text embeddings. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 1102–1121.

Weiwei Sun, Zhengliang Shi, Wu Long, Lingyong Yan, Xinyu Ma, Yiding Liu, Min Cao, Dawei Yin, and Zhaochun Ren. 2024. MAIR: A massive benchmark for evaluating instructed retrieval. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 14044–14067.

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems (NIPS)*, pages 5998–6008.

Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. 2023. Improving text embeddings with large language models. *arXiv*.

Orion Weller, Benjamin Chang, Sean MacAvaney, Kyle Lo, Arman Cohan, Benjamin Van Durme, Dawn J. Lawrie, and Luca Soldaini. 2025a. Followir: Evaluating and teaching information retrieval models to follow instructions. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 11926–11942.

Orion Weller, Benjamin Van Durme, Dawn J. Lawrie, Ashwin Paranjape, Yuhao Zhang, and Jack Hessel. 2025b. Promptriever: Instruction-trained retrievers can be prompted like language models. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-pack: Packaged resources to advance general chinese embedding. *arXiv*.

Xiang Zhang, Junbo Jake Zhao, and Yann LeCun. 2015. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems (NIPS)*, pages 649–657.

Wenjie Zhuo, Yifan Sun, Xiaohan Wang, Linchao Zhu, and Yi Yang. 2023. Whitenedcse: Whitening-based contrastive learning of sentence embeddings. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 12135–12148.

A Prompt Templates

A.1 Text Summarization

This prompt instructs the LLM to generate a concise summary based on a given instruction. It emphasizes extracting key points without unnecessary details and also specifies the output format at the end of the prompt to facilitate result parsing.

> Summarize the text based on the following instruction. The summary must focus on the instruction's key points and not exceed 10 words.
> **Instruction**: *{instruction}*
> **Text**: *{text}*
> **Required Format**: Summary: <summary>
> **Note**: Only output the summary in English starting with "Summary:", do not include any other text.

A.2 Label Generation

This prompt instructs the LLM to define a clear, precise category label that captures the core feature of the positive texts while distinguishing them from negative texts. It also specifies the output format at the end of the prompt to facilitate result parsing.

> Analyze these two groups of texts and define a clear category label that best describes the characteristics of the current group based on the following instructions.
> **Instruction**: *{instruction}*
> **Current Group Texts**: *{positive_texts}*
> **Other Group Texts**: *{negative_texts}*
> **Requirements**:
> - The label MUST strictly follow and reflect the given instruction.
> - Focus on the main characteristics of the current group based on the instruction.
> - Label should be generalizable but distinguishable from other texts.
> - Use clear and precise language.
> - The category name should be no more than 5 words.
> **Required Format**: Category: <category>
> **Note**: Only output the category name starting with "Category:", do not include any other text.

A.3 Text Classification

This prompt instructs the LLM to classify a given text based on the user instruction and predefined categories. It also specifies the output format at the end of the prompt to facilitate result parsing.

Downstream Task	Dataset	Instruction Focus	Category
Clustering	NYTClust	Topic	{Estate, Technology, Science, · · · , Sports}
		Location	{Germany, France, America, China, Canada, Britain, · · · , Japan}
	AmzCF	Counterfactual	{Counterfactual, Not-Counterfactual}
	MNews	Topic	{Business, Politics, Sports, Entertainment, Health, · · · , Technology}
STS	PaperCode	Method	{Gaussian Process, Q-Learning, SVM, CLIP, PCA, · · · , NeRF}
		Task	{Federated Learning, Active Learning, · · · , Question Answering}
	MultiHate	Hateful	{Hateful, Non-Hateful}
		Language	{Arabic, German, French, Italian, · · · , Mandarin}
	Big Patent	Patent Category	{Human Necessities, Physics, Electricity, Chemistry, · · · , Textiles}
Triplet Alignment	IntEmo	Intent	{Track Card, Inquiry Exchange Rates, · · · , Cash Withdrawal}
		Emotion	{Positive, Negative}
	Toxic	Toxic	{Toxic, Not Toxic}
	AG-News	Topic	{World, Sports, Business, Sci/Tech}

Table 6: The downstream tasks and their associated datasets. This benchmark involves diverse semantic aspects that can be specified in the user instructions, including topic, location, language, and intent.

Please classify the following text based on the instruction and available categories.
Instruction: *{instruction}*
Available Categories: *{categories}*
Text to Classify: *{text}*
Required Format: Classification: <category_name>
Note: Only output the category name starting with "Classification:", do not include any other text. The category must be exactly as listed above.

B Downstream Tasks and Datasets

Table 6 summarizes the downstream tasks and datasets used to evaluate instruction-aware embedding performance. Our benchmark spans three representative tasks, i.e., clustering, semantic textual similarity (STS), and triplet alignment, which are designed to assess how well embeddings reflect user-specified semantic intents.

The datasets cover a wide range of instruction-relevant aspects, including topic categories (e.g., AG-News, NYTClust), geographic locations (e.g., MNews), language types (e.g., MultiHate), and subjective attributes such as intent and emotion (e.g., IntEmo). These diverse tasks and semantic dimensions allow for a rigorous and comprehensive evaluation of how effectively GSTransform adapts pre-trained embeddings to align with different instruction-driven perspectives.

C Dataset-oriented Instructions

We provide the task instructions used by InstructOR, InBedder, and GSTransform for each dataset in Table 7. The baseline instructions adhere to their default stylistic conventions: InstructOR uses command-style prompts, while InBedder adopts a question-style format. Despite these stylistic differences, both formats convey the same semantic intent.

The instructions used in GSTransform largely mirror those of InBedder, with only minor deviations, primarily related to preferred response length, which are implicitly controlled through system-level prompts in InBedder. To ensure fair comparison and model consistency, we standardize these differences while preserving semantic equivalence across all methods.

D Full Results of Ablation Studies

Table 8 presents the full results of our ablation studies, evaluating the contribution of each core component in the GSTransform framework across all datasets. We assess the impact of (1) removing the instruction-based summarization step, (2)

Dataset	Aspect	Model	Instruction
NYTClust	Topic	InstructOR	Represent the text based on the main news category.
		InBedder	What is the main category of this news?
		Ours	What is the main category of this news?
	Location	InstructOR	Represent the text based on where the news happen.
		InBedder	Where did the news happen?
		Ours	What country does this news mention? Just tell me the country name.
AmzCF	Counterfactual	InstructOR	Represent the text based on whether it is counterfactual (yes/no).
		InBedder	Is the sentence counterfactual?
		Ours	Is the sentence counterfactual? just tell me yes/no.
MNews	Topic	InstructOR	Represent the text based on the main category of the news.
		InBedder	What is the main category of this news?
		Ours	What is the main category of this news?
PaperCode	Method	InstructOR	Represent the paper based on the method used.
		InBedder	What is the method used in this paper?
		Ours	What is the method used in this paper?
	Task	InstructOR	Represent the paper abstract based on the research task.
		InBedder	What is the research task of this paper abstract?
		Ours	What is the research task of this paper abstract?
MultiHate	Hateful	InstructOR	Represent the text based on whether it is hateful or not.
		InBedder	Is the text hateful? Just tell me yes/no.
		Ours	Is the text hateful? Just tell me yes/no.
	Language	InstructOR	Represent the text based on the language.
		InBedder	What is the language of the text? Just tell me the language.
		Ours	What is the language of the text? Just tell me the language.
Big Patent	Patent Category	InstructOR	Represent the text based on the category the patent belongs to according to the Cooperative Patent Classification (CPC) code.
		InBedder	What category does the patent belong to according to the Cooperative Patent Classification (CPC) code?
		Ours	What category does the patent belong to according to the Cooperative Patent Classification (CPC) code?
IntEmo	Intent	InstructOR	Represent the text based on what the customer needs.
		InBedder	What does the customer need?
		Ours	What does the user care about? Just tell me the name of the thing.
	Emotion	InstructOR	Represent the text based on the emotion of the user.
		InBedder	What is the emotion of the user?
		Ours	What is the emotion of the user? Just tell me the emotion.
Toxic	Toxic	InstructOR	Represent the text based on whether it is toxic (yes/no).
		InBedder	Is the sentence toxic? just tell me yes/no.
		Ours	Is the sentence toxic? just tell me yes/no.
AG-News	Topic	InstructOR	Represent the news according to their topic category.
		InBedder	What is the topic category of this news?
		Ours	What is the topic category of this news? Just tell me the category name.

Table 7: Instructions of InstructOR, InBedder, and GSTransform for each dataset.

Solutions	Clustering			STS			Triplet Alignment			Mean ↑
	NYTClust	AmzCF	MNews	PaperCode	MultiHate	Big Patent	IntEmo	Toxic	AG-News	
GSTransform (Mxbai)	73.92	34.14	64.71	83.43	52.45	38.34	96.30	63.53	87.27	66.01
Remove Summ.	68.32	38.85	64.14	81.72	16.21	34.81	85.14	57.55	86.47	59.25
Directed Label Gen.	42.50	14.60	18.24	30.04	20.55	26.97	86.76	55.06	74.30	41.00
FDA-based Transf.	67.53	35.77	63.15	75.15	48.49	21.56	89.19	57.81	74.10	59.19

Table 8: Full results of performance evaluations for different solutions in ablation studies.

replacing the full label construction process with direct LLM-generated labels, and (3) substituting our transformation model with an FDA baseline.

These results offer a more comprehensive view of how each design choice affects model performance, reinforcing the necessity of both the instruction-based label construction and the label-guided embedding transformation for achieving robust instruction-following behavior.

Data Size	InstructOR	InBedder-Roberta	GSTransform
1,000	12 s	7 s	36 s
3,000	28 s	19 s	68 s
10,000	74 s	60 s	68 s
30,000	173 s	175 s	69 s
100,000	450 s	583 s	70 s

Table 9: Real-time latency comparison across varied dataset sizes on AG-News.

Data Size	InstructOR	InBedder-Roberta	GSTransform
1,000	36 s	27 s	39 s
3,000	95 s	81 s	83 s
10,000	291 s	266 s	86 s
30,000	808 s	794 s	85 s
50,000	1,304 s	1,327 s	87 s

Table 10: Real-time latency comparison across varied dataset sizes on Big Patent.

E Efficiency Comparisons across Different Dataset Scales

We compare the real-time latency of GSTransform and baseline models across varying dataset sizes. As shown in Tables 9 and 10, baseline latency scales linearly with data volume, making them more suitable for small-scale scenarios. For instance, they remain competitive on the Big Patent dataset when the size is under 3,000 samples. In contrast, GSTransform achieves near-constant latency regardless of dataset size, as its transformation relies on a fixed-size sample. This scalability results in substantial efficiency advantages for large-scale applications.